Лабораторная работа №5 Интерфейсы. Контракт, callbacks, лямбда-выражения.

Интерфейс в Java — это контракт, который описывает поведение (набор методов), не реализуя его. Классы, использующие ключевое слово **implements**, обязуются реализовать все методы интерфейса. Интерфейсы частично решают проблему множественного наследования, в отличии от C++, где это разрешено, в Java можно унаследоваться только от одного класса, но реализовать может неограниченное количество интерфейсов. В отличие от абстрактного класса у интерфейса нету конструктора и по смыслу он нужен для определения поведения объекта, у абстрактного класса есть состояние, конструктор, также он может содержать как обычные, так и абстрактные метода помеченные ключевым слово **abstract**.

Создание интерфейса и его смысл

Мы опустим очевидные моменты и нырнем сразу в самую гущу событий и посмотрим как и в каких случаях их можно использовать. Для этого создадим интерфейс с названием **Filter,** он будет содержать метод **doFilter**, который будет что-то фильтровать, а кто это будет делать не важно:

```
public interface Filter {
    String[] doFilter(String[] str);
}
```

Далее создадим три класса: AuthFilter, RegFilter, RequestFilter. Они будут реализовывать наш ранее созданный интерфейс:

```
public class AuthFilter implements Filter {

    @Override
    public String[] doFilter(String[] str) {
        return null;
    }

}


public class RegFilter implements Filter {

    @Override
    public String[] doFilter(String[] str) {
        return null;
    }

}
```

```
public class RequestFilter implements Filter {

    @Override
    public String[] doFilter(String[] str) {
        return null;
    }

}
```

Пока эти классы оставим без реализации. А займемся обработчиком фильтров. Это будет класс с названием **FilterProcessor**, он будет содержать один единственный метод **process,** поле **filters** и конструктор с этим полем:

```
public class FilterProcessor {

    private Filter[] filters;

    public FilterProcessor(Filter[] filters) {
        this.filters = filters;
    }

    public String[][] process(String[] str) {
        String[][] filtered = new String[filters.length][];
        for (int i = 0; i < filters.length; i++) {
            filtered[i] = filters[i].doFilter(str);
        }
        return filtered;
    }

}
```

Метод **process** будет иметь следующую сигнатуру: String[][] process(String[] str). Для того, чтобы можно было накапливать результаты работы метода **doFilter,** используется двумерный массив строк.

Далее, реализация метода doFilter в классах: AuthFilter, RegFilter, RequestFilter. Смысл фильтра в данном случае - забрать из строки только, то что нужно, остальное отбросить. Допустим у нас есть такие массивы строк:

```
String[] req1 = {"nik: vasya123", "name: Vasya", "email: vasya@mail.com", "pass: ****"};
String[] req2 = {"head: 123456", "body: {name: Vasya, email: vasya@mail.com}"};
```

Чтобы забрать, например, значение ника и пароля, нужно разделить строку по символу двоеточия и взять, то что будет после него. Это можно сделать с помощью метода **split(":", 2),** первый параметр - разделитель, второй - сколько раз нужно разделить строку, в случае повторения разделителя, метод возвращает массив строк. После, в условии надо взять первый элемент из массива **credentials** и, если он является ником, то в массив записать значение, по аналогии поступаем с паролем.

```
public class AuthFilter implements Filter {

    @Override
    public String[] doFilter(String[] arr) {
        String[] filtered = new String[arr.length];
        for (int i = 0; i < arr.length; i++) {
            String[] credentials = arr[i].split(":", 2);
            if (credentials[0].equals("nik") || credentials[0].equals("pass") ) {
                filtered[i] = credentials[1];
            }
        }
        return filtered;
    }

}
```

По аналогии поступаем с остальными фильтрами:

```
public class RequestFilter implements Filter {

    @Override
    public String[] doFilter(String[] arr) {
        String[] filtered = new String[arr.length];
        for (int i = 0; i < arr.length; i++) {
            String[] credentials = arr[i].split(":", 2);
            if (credentials[0].equals("head") || credentials[0].equals("body")) {
                filtered[i] = credentials[1];
            }
        }
        return filtered;
    }

}


public class RegFilter implements Filter {

    @Override
    public String[] doFilter(String[] arr) {
        String[] filtered = new String[arr.length];
        for (int i = 0; i < arr.length; i++) {
            String[] credentials = arr[i].split(":", 2);
            if (credentials[0].equals("nik")
                    || credentials[0].equals("name")
                    || credentials[0].equals("email")
                    || credentials[0].equals("pass")) {
                filtered[i] = credentials[1];
            }
        }
        return filtered;
    }

}
```

Также создадим экземпляр **FilterProcessor** и в конструктор передадим набор фильтров и в цикле for-each выполним метод **process** и выедем все значения в консоль:

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        String[] req1 = {"nik: vasya123", "name: Vasya", "email: vasya@mail.com", "pass: ****"};
        String[] req2 = {"head: 123456", "body: {name: Vasya, email: vasya@mail.com}"};

        String[][] reqs = {req1, req2};

        FilterProcessor processor = new FilterProcessor(new Filter[] {
                new RegFilter(),
                new AuthFilter(),
                new RequestFilter()
                });

        for (String[] req : reqs) {
            String[][] arrs = processor.process(req);
            for (String[] arr: arrs) {
                System.out.println(Arrays.toString(arr));
            }
        }
    }
}
```

В консоли будет следующее:

```
[ vasya123,  Vasya,  vasya@mail.com,  ****]
[ vasya123, null, null,  ****]
[null, null, null, null]
[null, null]
[null, null]
[ 123456,  {name: Vasya, email: vasya@mail.com}]
```

Некоторые значения, а также массивы принимают null. Это происходит из-за особенностей реализации метода doFilter в классах, т.к. массивы строк принимают по-умолчанию null и длина массива в паре методов больше чем в него записано значений, то в результате у нас значения null. Можно улучшить алгоритм и избавится от дублирования кода, создав отдельный класс и задать длину массива равную количеству параметров фильтра.

В результате код придерживается принципов ООП, благодаря интерфейсу код стал гибким и расширяемым. Если мы добавим еще десять фильтров, нам не нужно переписывать основную логику программы. В этом и есть удобство использования полиморфизма.

Механизм обратного вызова (callback)

Обратный вызов — это передача функции (метода) в качестве аргумента, чтобы её можно было вызвать позднее. В Java, где нет функций как первоклассных объектов, эту роль выполняют интерфейсы с определёнными методами. Суть обратного вызова состоит в том, что мы создаем действия, которые вызываются при других действиях. То есть одни действия вызываются другими действиями. Классический пример - нажатие кнопки. Когда мы

нажимаем на кнопку, мы производим действие, но в ответ на это нажатие запускаются другие действия. Обычно для создания интерфейса, который будет использоваться в качестве события, используют один метод, реже - два.

Создадим интерфейс **EventListener** с одним методом **onEvent:**

```
interface EventListener {
      void onEvent();
}
```

Также создадим класс, генерирующий события:

```
class EventManager {
    private EventListener listener;

    public void setEventListener(EventListener listener) {
        this.listener = listener;
    }

    public void simulateEvent() {
        System.out.println("Событие произошло!");
        if (listener != null) {
            listener.onEvent();
        }
    }
}
```

Класс реагирующий на событие, реализует контракт, также его можно назвать делегатом как в C#:

```
class AlertSystem implements EventListener {
    @Override
    public void onEvent() {
        System.out.println("AlertSystem: Реагирую на событие!");
    }
}
```

Создадим экземпляры классов EventManager и AlertSystem:

```
public class Main {
    public static void main(String[] args) {
        EventManager manager = new EventManager();
        AlertSystem delegate = new AlertSystem();

        manager.setEventListener(delegate);
        manager.simulateEvent();
    }
```

```
}
```

Результат выполнения программы:

```
Событие произошло!
AlertSystem: Реагирую на событие!
```

В этом примере мы имитируем вызов события, на практике интерфейсы в Java предоставляют элегантный способ реализации обратных вызовов, позволяя создавать гибкие и легко расширяемые архитектуры. Этот механизм используется в GUI (Graphical user interface), асинхронном программировании.

В GUI, тут мы подписались на событие нажатия кнопки:

```
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Кнопка нажата!");
    }
});
```

Лямбда-выражения

Java задумывался как полностью объектно ориентированный язык программирования, функции сами по себе не могу существовать отдельно от объектов в отличии, например, от JavaScript, где функции выходят на передний план. Но в восьмой версии Java появилось множество нововведений, включая лямбда-выражения.

Лямбда-выражения в Java - это единственный метод интерфейса без его объявления, анонимная функция. За счет этого наш код сокращается, по сути нам остается написать только его реализацию и аргументы. Синтаксис лямбд следующий:

(аргументы) -> { реализация }

Далее идет несколько примеров лямбда-выражений:

```
() -> System.out.println("Hello world");

it -> it.name;

(int a, int b) -> a + b

soundPool.setOnLoadCompleteListener((soundPool, sampleId, status) ->
isComplete = true);

buttonReset.setOnClickListener(v -> {
   if (mService != null) {
       mService.reset();
       stopTimer();
```

```
    }
});
```

Интерфейс с одним методом без реализации, который используется в лямбда-выражениях - это функциональные интерфейсы. Они обычно помечаются аннотацией `@FunctionalInterface`, которая обозначает, что этот интерфейс является функциональным. На самом деле главное условие, чтобы интерфейс являлся функциональным - это чтобы он имел единственный метод без реализации (см. рис. 1).

```
@FunctionalInterface
public interface Function<T, R> {
    R apply(T var1);
```

рис. 1 Функциональный интерфейс

* Примечание 1. Символы T и R являются обобщенными типами (дженериками), будут рассматриваться в следующих лабораторных.

Общие функциональные интерфейсы в Java:

1. *Predicate<T>:* Принимает аргумент типа T и возвращает логическое значение. Обычно используется для фильтрации данных.

   ```
   Predicate<String> nonEmptyStringPredicate = s -> !s.isEmpty();
   ```

2. *Function<T, R>:* Принимает аргумент типа T и возвращает результат типа R. Используется для преобразования данных.

   ```
   Function<String, Integer> stringLengthFunction = s -> s.length();
   ```

3. *Consumer<T>:* Принимает аргумент типа T и не возвращает результат (void). Используется для выполнения действий над объектами.

   ```
   Consumer<String> printConsumer = s -> System.out.println(s);
   ```

4. *Supplier<T>:* Представляет поставщика (supplier) в отличии от потребителя (consumer) предоставляет результатов без входных аргументов.

   ```
   Supplier<Long> currentTimeSupplier = () -> System.currentTimeMillis();
   ```

Мы можем сами создать лямбда-выражения:

```java
public class Test {
    public static void main(String[] args) {
        Action action = () -> {
            System.out.println("Hello world"); //  реализация анонимной функции
        };

        action.apply(); // вызов метода
    }

    interface Action { // функциональный интерфейс
        void apply(); // единственный метод
    }
```

Для этого напишем интерфейс Action с единственным методом apply(), который ничего не будет принимать и ничего не будет возвращать. В методе main создадим реализацию этого интерфейса, но будем использовать для этого анонимную ф-ю. В теле напишем, что мы хотим делать, например, вывести Hello world в консоль. В конце вызовем apply, чтобы выполнить действия. Также можем вынести вызова метода функционального интерфейса:

```java
public class Test {
    public static void main(String[] args) {
        // вызов метода
        toAction(() -> System.out.println("Hello world"));
    }

    interface Action { // функциональный интерфейс
        void apply(); // единственный метод
    }

    // метод-обертка для вызова функции
    public static void toAction(Action action) {
        action.apply();
    }
}
```

Интерфейс можно передать в качестве аргумента в конструктор и использовать лямбда-выражение при создании объекта:

```java
public class Test {
    public static void main(String[] args) {
//        // вызов метода
//        toAction(() -> System.out.println("Hello world"));
        new A(() -> System.out.println("Hello world"));
    }

    static class A {
        public A(Action a) {
            a.apply();
        }
    }
```

Давайте попробуем использовать лямбда-выражения в следующем коде:

```java
public String toString() {
    StringBuilder str = new StringBuilder(); // Создаем StringBuilder для записи строк

    // если CHAR, то одинарные кавычки, если STRING двойные
    String symbol = validate(type);

    for (Modificator m : modificaters) { // for-each для перебора списка модификаторов
        str.append(m.getValue()).append(" ");
    }

    str.append(type.getValue()).append(" ").append(name);
    if (!value.equals("")) {
        // условие, если значение переменной не пустое,
        // то прописываем оператор присваивания
        str.append(" = ").append(symbol).append(value).append(symbol);
    }
    return str.toString();
}
```

* Примечание 2. StringBuilder - класс, который похож на String, но более эффективный при конкатенации строк, его лучше использовать в циклах или где происходит множественное сложение строк.

Это код прошлой работы, в нем мы собираем наше поле из модификаторов, названия, типа и значения. Попробуем написать реализацию for each цикла с помощью лямбд:

```java
public void forEach(List<Modificator> list, Consumer<Modificator> action) {
    for (Modificator element : list) {
        action.accept(element);
    }
}
```

* Примечание 3. List - интерфейс из Java Collections Framework, имеет несколько реализаций, одна из которых - ArrayList, в отличии от массива автоматически расширяется. Будет рассмотрен подробнее в дальнейшем.

Для этого создадим метод forEach и в качестве аргументов он будет принимать список и интерфейс Consumer. У этого интерфейса существует единственный метод - accept (см. рис. 2)

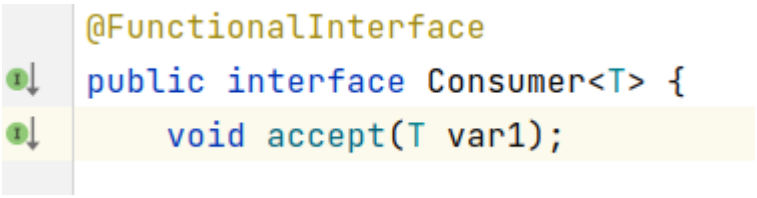


рис. 2 Функциональный интерфейс Consumer

В итоге мы можем использовать наш метод с лямбда-выражением:

```
public String toString() {
    StringBuilder str = new StringBuilder(); // Создаем StringBuilder для записи строк

    // если CHAR, то одинарные кавычки, если STRING двойные
    String symbol = validate(type);

    // использование нашего метода
    forEach(modificaters, value -> str.append(value.getValue()).append(" "));

    str.append(type.getValue()).append(" ").append(name);
    if (!value.equals("")) {
        // условие, если значение переменной не пустое,
        // то прописываем оператор присваивания
        str.append(" = ").append(symbol).append(value).append(symbol).append(endOfString);
    } else {
        str.append(endOfString);
    }

    return str.toString();
}
```

Зачастую уже существует метод, который делает именно то, что вы хотите сделать в другом месте. Например, предположим, что вы просто хотите распечатать все строки из списка.

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.forEach(name -> System.out.println(name));
```

Было бы лучше, если бы вы могли просто передать метод println в метод forEach. Примерно так:

```
names.forEach(System.out::println);
```

Выражение

```
name -> System.out.println(name)
```

эквивалентно

`System.out::println` - является ссылкой на метод println.

Еще примеры ссылок на методы:

```
Function<String, Integer> stringLengthFunction = s -> s.length();
Function<String, Integer> stringLengthFunction = String::length;

Supplier<Long> currentTimeSupplier = () ->
System.currentTimeMillis();
Supplier<Long> currentTimeSupplier = System::currentTimeMillis;

Arrays.sort(strs, String::compareToIgnoreCase)
```

Задания

Задания выбирать: номер в списке группы mod 4

19 mod 4 = 3

Нужно скопировать код из репозитория (ссылка ниже) по своему номеру задания, посмотреть, потренироваться и на паре реализовать эти методы при преподавателе, а также одно секретное задание того же типа до 10 минут на всё, без использования любых подсказок, в частности: записи в тетради или листочках или электронном блокноте, мобильный телефон, GPT-модели, Интернет. Возможны вопросы или внесение дополнительных изменений в код.

Задание 0. Один if внутри for
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var_n_0

Задание 1. Один вызов метода
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var_n_1

Задание 2. Мат. Оператор
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var_n_2

Задание 3. Одна строка в main
https://github.com/vadhub/MacroLabs-for-Students/tree/main/var_n_3

Критерии оценки

Сделал	Оценка
Основной TODO 1 + TODO 2 работают	зачёт
Секретное задание тоже работает	бонус к оценке
Только TODO 1 работает	не зачёт
Основной не работает	не зачёт
Не написал ничего за 10 минут	не зачёт